

Festiwal

Nayra jest na festiwalu i gra w grę, w której główną nagrodą jest wycieczka do Laguna Colorada. Gra polega na nabywaniu kuponów za pomocą żetonów. Nabycie kuponu może skutkować zwielokrotnieniem liczby żetonów pozostałych po transakcji. Celem jest pozyskanie jak największej liczby kuponów.

Rozpoczyna grę z A żetonami. Dostępnych jest N kuponów, ponumerowanych od 0 do $N - 1$. Nayra musi wydać $P[i]$ żetonów ($0 \leq i < N$), aby nabyć kupon i (i musi mieć przynajmniej $P[i]$ żetonów przed transakcją). Nayra może nabyć każdy kupon co najwyżej raz.

Co więcej, każdy kupon i ($0 \leq i < N$) ma przypisany **typ**, oznaczony przez $T[i]$, który jest liczbą całkowitą **z przedziału od 1 do 4 włącznie**. Po tym, jak Nayra kupi kupon i , pozostała liczba posiadanych przez nią żetonów zostanie pomnożona przez $T[i]$. Formalnie, jeśli w pewnym momencie gry ma X żetonów, i kupi kupon i (co wymaga $X \geq P[i]$), to po zakupie będzie miała $(X - P[i]) \cdot T[i]$ żetonów.

Twoim zadaniem jest wyznaczenie, które kupony powinna kupić Nayra i w jakiej kolejności, aby zmaksymalizować sumaryczną liczbę **kuponów**, które będzie miała na samym końcu. Jeśli istnieje więcej niż jedna sekwencja zakupów, która pozwala osiągnąć ten cel, możesz podać dowolną z nich.

Szczegóły implementacji

Powineneś zaimplementować następującą procedurę.

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
                             std::vector<int> T)
```

- A początkowa liczba żetonów posiadanych przez Nayrę.
- P tablica długości N opisująca ceny kuponów.
- T tablica długości N opisująca typy kuponów.
- Ta procedura jest wywoływana dokładnie raz dla każdego przypadku testowego.

Procedura powinna zwrócić tablicę R , która opisuje zakupy Nayry w następujący sposób:

- Długość tablicy R powinna być maksymalną liczbą kuponów, które może kupić.

- Elementy tablicy są indeksami kuponów, które powinna kupić, w porządku chronologicznym. Oznacza to, że najpierw kupuje kupon $R[0]$, następnie kupon $R[1]$ i tak dalej.
- Elementy R muszą być parami różne.

Jeśli nie można kupić żadnych kuponów, R powinna być pustą tablicą.

Ograniczenia

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ dla każdego i takiego, że $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ dla każdego i takiego, że $0 \leq i < N$.

Podzadania

Podzadanie	Punktacja	Dodatkowe ograniczenia
1	5	$T[i] = 1$ dla każdego i takiego, że $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ dla każdego i takiego, że $0 \leq i < N$.
3	12	$T[i] \leq 2$ dla każdego i takiego, że $0 \leq i < N$.
4	15	$N \leq 70$
5	27	Nayra może kupić wszystkie N kupony (w pewnej kolejności).
6	16	$(A - P[i]) \cdot T[i] < A$ dla każdego i takiego, że $0 \leq i < N$.
7	18	Brak dodatkowych ograniczeń.

Przykłady

Przykład 1

Rozważmy następujące wywołanie.

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Nayra początkowo posiada $A = 13$ żetonów. Może kupić 3 kupony w kolejności pokazanej poniżej:

Kupiony kupon	Cena kuponu	Typ kuponu	Liczba żetonów po transakcji
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

W tym przykładzie Nayra nie może kupić więcej niż 3 kuponów, a opisana powyżej sekwencja zakupów jest jedynym sposobem, w jaki może kupić 3 kupony. W związku z tym procedura powinna zwrócić $[2, 3, 0]$.

Przykład 2

Rozważmy następujące wywołanie.

```
max_coupons(9, [6, 5], [2, 3])
```

W tym przykładzie Nayra może kupić oba kupony w dowolnej kolejności. W związku z tym procedura powinna zwrócić $[0, 1]$ lub $[1, 0]$.

Przykład 3

Rozważmy następujące wywołanie.

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

W tym przykładzie Nayra ma jeden żeton, który nie wystarcza na kupno żadnego kuponu. Dlatego procedura powinna zwrócić $[]$ (pustą tablicę).

Przykładowy program oceniający

Format wejściowy:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Format wyjściowy:

```
S
R[0] R[1] ... R[S-1]
```

Gdzie S jest długością tablicy R zwróconej przez `max_coupons`.